

anode*Positive terminal of a polarized component (LED, battery, etc)***cathode***Negative terminal of a polarized component (LED, battery, etc)*

The GPIO port outputs are digital and do not offer such fine-grain voltage control. They can provide two levels: almost ground, and almost V_{DD} (e.g. 3.3 V). The first will correctly keep the LED off, but the second will probably burn out the LED. We need to include resistor R2 to limit the current through the LED and MCU driver output to a safe value.

We can calculate the value R of a current-limiting resistor based on the supply voltage V_{DD} , the LED forward voltage V_F , and the desired LED current I_{LED} :

$$R = \frac{V_{DD} - V_F}{I_{LED}}$$

We start by picking a value of I_{LED} which is safe for both the LED and the MCU output. In this case let's set $I_{LED} = 4$ mA and assume $V_{DD} = 3.0$ V. V_F is 1.8 V for the red LED and 2.7 V for the blue LED. We can now solve for the resistor values. For the red LED, a 300 Ω resistor is needed. For the blue LED, a 75 Ω resistor is needed.

Inside the MCU

Now that we know how to connect these simple devices to the MCU pins, let's look into how to let the program running on the CPU talk with those pins. First the program needs to configure the hardware within the MCU to set up the path between the CPU and the pins. Then the program can read from or write to those pins. Different MCU families use different approaches; here we cover the STM32F0 MCU family.

Preliminaries: Control Registers and C Code

Peripherals are built with **control registers** to allow us to configure them, determine their status and transfer data. Table 2.1 shows some of the control registers for one peripheral module, the Reset and Clock Controller (RCC). These control registers appear as special locations in memory, as indicated by the column marked "Absolute address (hex)". The term "hex" indicates the address is in **hexadecimal** format (base 16). We also can see the width of the register and the value after the MCU is reset (a value of X indicates that the corresponding bits are undefined). The reference manual offers further information [2].

The MCU's reference manual provides the diagram in Figure 2.5 for RCC_AHBENR, showing how each bit is used (if used), and whether we can read from or write to it. The manual also explains the use of each field in more detail.

control register

Register used to configure operation of hardware in CPU or peripheral

hexadecimal

Base-sixteen numbering system. Each digit can have one of sixteen values (0 through 9, A, B, C, D, E and F). Symbols A through F represent values of ten through fifteen.

Table 2.1 A portion of registers for the Reset and Clock Controller (RCC) peripheral. Registers are named with the peripheral name (RCC) followed by the register name (_CR) [2].

Absolute Address (hex)	Register name	Width (bits)	Reset value	Section/ page
4002_1000	Clock control register (RCC_CR)	32	0000_XX83h	6.4.1/108
4002_1004	Clock configuration register (RCC_CFGR)	32	0000_0000h	6.4.2/110
4002_1008	Clock interrupt register (RCC_CIR)	32	0000_0000h	6.4.3/113
4002_100C	APB peripheral reset register 2 (RCC_APB2RSTR)	32	0000_0000h	6.4.4/116
4002_1010	APB peripheral reset register 1 (RCC_APB1RSTR)	32	0000_0000h	6.4.5/117
4002_1014	AHB peripheral clock enable register (RCC_AHBENR)	32	0000_0014h	6.4.6/120
4002_1018	APB peripheral clock enable register 2 (RCC_APB2ENR)	32	0000_0000h	6.4.7/121
4002_101C	APB peripheral clock enable register 1 (RCC_APB1ENR)	32	0000_0000h	6.4.8/123
4002_1020	RTC domain control register (RCC_BDCR)	32	0000_0018h	6.4.9/126
4002_1024	Control status register (RCC_CSR)	32	XXX0_0000h	6.4.10/128

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Field	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TSCEN	Res.	IOPF EN	IOPE EN	IOPD EN	IOPC EN	IOPB EN	IOPA EN	Res.
Reset								0		0	0	0	0	0	0	
Access								rw		rw	rw	rw	rw	rw	rw	

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CRC EN	Res.	FILTF EN	Res.	SRAM EN	DMA2 EN	DMA EN
Reset										0		1		1	0	0
Access										rw		rw		rw	rw	rw

Figure 2.5 Control register RCC_AHBENR allows enabling clock signal to peripherals on AHB bus. From Section 6.4.6 of MCU reference manual [2].

Using CMSIS to Access Hardware Registers with C Code

It would be tedious to have to look up and remember the addresses for the hardware control registers. Instead we use special C-language support. The Cortex Microcontroller Software Interface Standard (CMSIS) is a hardware abstraction layer for Cortex-M processors. The CMSIS-CORE component provides a C-language interface to the processor core and peripherals. This consists of macros and functions to perform various operations, and C data structures that map directly to registers.

Consider the `RCC_AHBENR` control register. It is one of the RCC peripheral's control registers. CMSIS-CORE lets us access the RCC control registers using a C-language data structure with a useful name (`RCC`). The data structure for the RCC peripheral contains 32-bit fields called `CR`, `CFGR`, `CIR`, and so forth. To access the `RCC_AHBENR` register, we simply write `RCC ->AHBENR`. Note that `RCC` is defined as a pointer to a data structure, which is why we use the `->` to select the control register within.

There are similar data structures for all of the MCU's peripherals and their control registers. The file `stm32f091xc.h` defines the device peripheral access layer (DPAL) for CMSIS-CORE for STM32F091xC-type MCUs. We need to be sure that all of our C source files contain the directive `#include <stm32f091xc.h>` before we try to use these features.

Beyond the device peripheral access layer (DPAL), ST provides additional software support modules designed to simplify application development. These include low-layer (LL) drivers and the STM32Cube Hardware Abstraction Layer (HAL). Please refer to this text's preface for a discussion.

Coding Style for Accessing Bits

A control register may hold one or more items of information (e.g. a count of how many pulses have been received). Each item is called a "field" and has a width of one or more bits. Consider the register `RCC_AHBENR` shown in Figure 2.5. This is a 32-bit register. Bit 0 is a one-bit field labeled `DMAEN`, and bit 1 is one-bit field labeled `DMAEN2`. Bit 3 and bit 5 are reserved fields and should not be accessed. Bit 4 is a field labeled `FLITFEN`. Bit 6 is a field labeled `CRCEN`. Bits 7 to 16 and bits 23 to 31 are reserved and should not be accessed. Bits 17 to 22 are labeled `IOPAEN`, `IOPBEN`, `IOPCEN`, `IOPDEN`, `IOPEEN`, `IOPFEN` [2].

How can we access fields in these control registers using C code? We often need to access one or more specific bits in a control register to set them to specific values. For example, we might need to set the fields `IOPAEN` and `IOPBEN` to 1 in `RCC_AHBENR`.

The fields are located at bits 17 and 18, respectively. We could write a 32-bit value with those bits set: the binary representation is 0000 0000 0000 0110 0000 0000 0000 0000. In decimal this is $2^{17} + 2^{18} = 131072 + 262144$. However, it is slow and tedious counting zeros and we are likely to make errors. Furthermore, the meaning of the code (shown below) is not at all clear.

```
n = 393216;
```

We can make the code somewhat easier to write and maintain by forming the value as a sum of shifted one bits. We'll use the left shift operation `a << b`, which shifts operand `a` to the left by `b` bit positions. We will also use the bitwise or operator `|` rather than addition.

```
n = (1UL << 17) | (1UL << 18);
```

Why is there “UL” after the 1? To write to a 32-bit register we need for the compiler to generate a 32-bit unsigned integer. To ensure the compiler does not use a signed or short integer and introduce possible errors, we use the suffix UL to specify that a numeric literal (e.g. 1) should be represented as an unsigned long integer, which is 32 bits long for an Arm Cortex-M processor.

We can make the code even easier to understand if we define and use meaningful names for the bit positions. When we use them, our code now becomes much easier to read:

```
#define MY_IOPAEN_POS (17)
#define MY_IOPBEN_POS (18)
n = (1UL << MY_IOPAEN_POS) | (1UL << MY_IOPBEN_POS);
```

Let’s further simplify things by making a macro to create a mask value by shifting a 1 to the proper position:

```
#define MASK(x) (1UL << (x))
n = MASK(MY_IOPAEN_POS) | MASK(MY_IOPBEN_POS);
```

MCU vendors typically provide files which define convenient names for fields, using a <peripheral type>_<register>_<field> naming convention. For example, Listing 2.1 shows some of the definitions related to the RCC’s AHBENR register and the GPIO’s MODER register. The file defines three symbols for each bit field:

- The symbol ending in **_Pos** indicates the position of the least-significant bit of the field within the register. For example, **RCC_AHBENR_FLITFEN_Pos** is 4.
- The symbol ending in **_Msk** has ones in the field’s bit positions within the register, and zeroes elsewhere. For example, in binary **RCC_AHBENR_FLITFEN_Msk** is 00000000 00000000 00000000 00010000.
- The final symbol (with no suffix) is set to be the same as the corresponding **_Msk** field and is used to improve code readability (as we’ll see shortly).

```
/****** Bit definition for RCC_AHBENR register *****/
// ...
#define RCC_AHBENR_FLITFEN_Pos    (4U)
#define RCC_AHBENR_FLITFEN_Msk    (0x1U << RCC_AHBENR_FLITFEN_Pos)
#define RCC_AHBENR_FLITFEN      RCC_AHBENR_FLITFEN_Msk
#define RCC_AHBENR_CRCEN_Pos     (6U)
#define RCC_AHBENR_CRCEN_Msk     (0x1U << RCC_AHBENR_CRCEN_Pos)
#define RCC_AHBENR_CRCEN        RCC_AHBENR_CRCEN_Msk
#define RCC_AHBENR_GPIOAEN_Pos   (17U)
#define RCC_AHBENR_GPIOAEN_Msk   (0x1U << RCC_AHBENR_GPIOAEN_Pos)
#define RCC_AHBENR_GPIOAEN      RCC_AHBENR_GPIOAEN_Msk
#define RCC_AHBENR_GPIOBEN_Pos   (18U)
#define RCC_AHBENR_GPIOBEN_Msk   (0x1U << RCC_AHBENR_GPIOBEN_Pos)
#define RCC_AHBENR_GPIOBEN      RCC_AHBENR_GPIOBEN_Msk
// ...

/****** Bit definition for GPIO_MODER register *****/
#define GPIO_MODER_MODER0_Pos    (0U)
#define GPIO_MODER_MODER0_Msk    (0x3U << GPIO_MODER_MODER0_Pos)
#define GPIO_MODER_MODER0      GPIO_MODER_MODER0_Msk
// ...
```

```
#define GPIO_MODER_MODER1_Pos    (2U)
#define GPIO_MODER_MODER1_Msk    (0x3U << GPIO_MODER_MODER1_Pos)
#define GPIO_MODER_MODER1        GPIO_MODER_MODER1_Msk
// ...
```

Listing 2.1 Some examples of bit field definitions for CMSIS-CORE device peripheral abstraction layer in `stm32f091xc.h`. These follow the naming convention of `<peripheral type>_<register>_<field>`.

Note that the name defined for the IOPAEN fields is **GPIOAEN**, since IOPAEN is at bit position 17, and **RCC_AHBENR_GPIOAENR_Pos** is defined to be 17. In fact, all six of the IOP fields follow this convention. This is an exception; usually the names for bit field definitions match the reference manual. When in doubt, examine the header file (e.g. `stm32f091xc.h`) and find the name defined with the **_Pos** value matching the start of the field in question.

Also, note that the **RCC_AHBENR_Msk** definitions are shifting a single “one” bit (0x1U), because each field is a single bit wide. However, the **GPIO_MODER_Msk** definitions are shifted values of 3. This is because this field is two bits wide and requires shifting two “ones” (00000011 binary, or 0x03 in hex).

CMSIS-CORE provides the macros **_VAL2FLD** and **_FLD2VAL** for converting between values and bit fields. These are shown in Listing 2.2, excerpted from the file `core_cm0.h`. These macros are especially useful for fields which are more than one bit wide (our **MASK** macro would need to be modified to handle more than one bit).

The **_VAL2FLD** macro takes the value argument (which is aligned so the LSB is at bit position 0), shifts it left to the correct field position, and then performs a logical AND to keep only the field’s bits, zeroing out the others. The **_FLD2VAL** macro takes a value argument (which is now aligned so the LSB is at the **_Pos** bit position), performs a logical AND to keep only the field’s bits (zeroing out the others), and then shifts the result to the right so the LSB is at bit position 0.

The compiler’s preprocessor stage takes the field argument and uses the **##** operator to concatenate the field name with a suffix of **_MSK** or **_POS** to create a macro which uses the correct names.

```
#define _VAL2FLD(field, value) \
(((uint32_t)(value) << field ## _Pos) & field ## _Msk)

#define _FLD2VAL(field, value) \
(((uint32_t)(value) & field ## _Msk) >> field ## _Pos)
```

Listing 2.2 Using CMSIS-CORE macro to shift values to or from their field locations. The `\` character tells the compiler to append the next line to the current line.

Reading, Modifying, and Writing Fields in Control Registers

Now we can see how to use these pieces to access the fields in the control registers. Note that these methods will work for fields which are a single bit or multiple bits wide.

- How do we find the current value of the GPIOAEN field? We use **_FLD2VAL** to read the control register AHBENR and extract the desired field:

```
// Find current value of GPIOAEN
n = _FLD2VAL(RCC_AHBENR_GPIOAEN, RCC->AHBENR);
```

- How do we set all the bits in the fields GPIOAEN and GPIOEEN in that register, leaving everything else as zero? We use the `=` assignment operator:

```
// Set GPIOAEN and GPIOEEN to all ones, clear others
RCC->AHBENR = RCC_AHBENR_GPIOAEN | RCC_AHBENR_GPIOEEN;
```

- How do we *set* all the bits in the fields GPIOAEN and GPIOEEN in that register without modifying anything else? This means we need to perform a read/modify/write operation. We read the initial control register value, modify the value, and then write the result back to the register. The OR read/modify/write operator `|=` does this:

```
// Set GPIOAEN and GPIOEEN to all ones, leave others unchanged
RCC->AHBENR |= RCC_AHBENR_GPIOAEN | RCC_AHBENR_GPIOEEN;
```

- How do we *clear* all the bits in the field GPIOAEN without modifying anything else? Again we need to perform a read/modify/write operation. However, the modification involves zeroing out the bit for GPIOAEN. We do this by first complementing the mask for GPIOAEN using the `~` operator. This flips all of its ones to zeros and zeros to ones. Using the AND read/modify/write operator `&=` will zero out the control register's bits for IO port A's field:

```
// Clear GPIOAEN to all zeroes, leave others unchanged
RCC->AHBENR &= ~RCC_AHBENR_GPIOAEN;
```

set

To change a bit to one

clear

To change a bit to zero

- How do we change a field with *multiple bits* without changing any other fields? This means we need to perform a read/modify/write operation. We read the initial control register value, modify the value, and then write the result back to the register. The OR read/modify/write operator `|=` does this. There are no multi-bit fields in the RCC AHBENR register, so let's consider the GPIOA peripheral's MODER register instead. That register has a two-bit wide field for each of GPIOA's 16 port bits, such as `GPIOA_MODER_MODER6` for bit 6. If we want to change that field to `n` we can use this code:

```
// Update a multibit field's value
// Clear all bits in field
GPIOA->MODER &= ~GPIOA_MODER_MODER6;
// Set bits in field according to shifted value
GPIOA->MODER |= _VAL2FLD(GPIOA_MODER_MODER6, n);
```

We will use this type of code frequently, so we will create a generalized, reusable macro to modify a field's value. The resulting code will be shorter and more concise, making it easier to understand (as well as write). We define this macro in a file called `field_access.h`, located in the common-driver directory of the code repository.

```
#define MODIFY_FIELD(reg, field, value) \
((reg) = ((reg)& ~ (field ## _Msk)) | \
(((uint32_t)(value) << field ## _Pos)& field ## _Msk))
```

The resulting code is simple and clear:

```
MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER5, n);
```

Configuring the I/O Path

Figure 2.6 shows an overview of the hardware circuits between an I/O pin and the CPU for a STM32F0 MCU. The GPIOA module selects which peripheral modules (e.g. GPIO, USART2, TIM2) will use port A's pins. Pins can be assigned individually to different peripherals. The MCU configures the peripheral module and exchanges data with it. The block labeled RCC provides a clock signal to only the active modules, reducing power consumption.

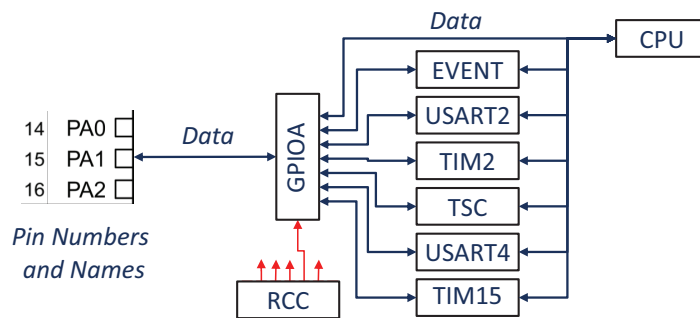


Figure 2.6 An overview of hardware between pin PA1 and the CPU.

In order to use the GPIO peripheral, we need to do a little preparation. First, we need to ensure that a clock signal is provided to the port module or else it will not operate. Second, we need to ensure that the GPIO signals inside the integrated circuit are routed to the outside world through an I/O pin (or pad).

Clock Gating

In STM32F0 MCUs, the RCC controls system **clock gating** using control registers AHBENR, APB1ENR and APB2ENR to control the peripherals. Chapter 6 in the MCU reference manual provides full information [2]. Figure 2.5 shows an excerpt from the manual. AHBENR allows individual control of the clock signals to GPIOA, GPIOB, GPIOC, GPIOD, GPIOE and GPIOF. Setting an I/O port's bit, IOPA, IOPB, IOPC, IOPD, IOPE and IOPF respectively, in this register

to one will supply the clock, allowing the port to operate. A zero will disable (gate) the clock signal to reduce power consumption by preventing circuit switching. We use the CMSIS-CORE support in the `stm32f091xc.h` header file:

```
RCC->AHBENR |= RCC_AHBENR_GPIOAEN;
```

clock gating

Method to disable circuit by blocking clock signal, reducing power consumption

Connecting a Pin to a Peripheral Module

The GPIOA through GPIOF modules provide control for each port's pins. Each I/O port x has four 32-bit configuration registers. To allow individual bit configuration of each pin, some bits are available in each configuration register. Table 2.2 shows the control registers for GPIOA.

Table 2.2 An example of GPIOA port control registers for a STM32F0 microcontroller.

Address	Register Description and Name	Width (bits)	Reset Value
4800_0000	GPIO port mode register (GPIOA_MODER)	32	2800_0000h
4800_0004	GPIO port output type register (GPIOA_OTYPER)	32	0000_0000h
4800_0008	GPIO port output speed register (GPIOA_OSPEEDR)	32	0000_0000h
4800_000C	GPIO port pull-up/pull-down register (GPIOA_PUPDR)	32	0C00_0000h
4800_0010	GPIO port input data register (GPIOA_IDR)	32	0000_XXXXh
4800_0014	GPIO port output data register (GPIOA_ODR)	32	0000_0000h
4800_0018	GPIO port bit set/reset register (GPIOA_BSRR)	32	0000_0000h
4800_001C	GPIO port configuration lock register (GPIOA_LCKR)	32	0000_0000h
4800_0020	GPIO port alternate function low register (GPIOA_AFR1)	32	0000_0000h
4800_0024	GPIO port alternate function high register (GPIOA_AFR2)	32	0000_0000h
4800_0028	GPIO port bit reset register (GPIOA_BRR)	32	XXXX_0000h

MCUs often include a **multiplexer**, which is an electronic selector switch. As shown in Figure 2.7, this allows a specific pin on the IC package to be connected to one of several possible peripheral modules. This provides greater flexibility to PCB designers and reduces MCU package size, circuit board size, and costs.

multiplexer

Electronic selector switch which routes one of N inputs signals to the output. MCU pin multiplexer is bidirectional (includes demultiplexer).

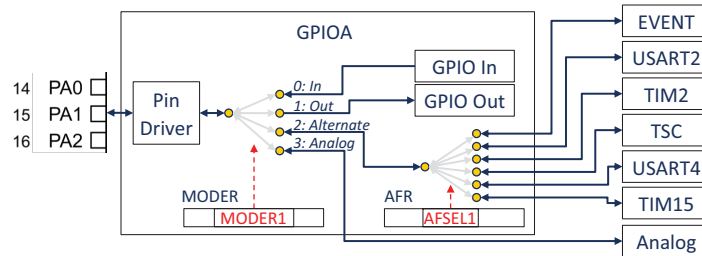


Figure 2.7 A detailed view of a pin multiplexer.

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Field	MODER15		MODER14		MODER13		MODER12		MODER11		MODER10		MODER9		MODER8	
Reset	0	0	x	0	x	0	0	0	0	0	0	0	0	0	0	0
Access	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	MODER7		MODER6		MODER5		MODER4		MODER3		MODER2		MODER1		MODER0	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Access	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 2.8 GPIO Mode Register controls first multiplexer to define mode for each GPIO pin.

The STM32F0 GPIOx modules provide two such multiplexers for each pin. The multiplexer closest to the pin determines whether the pin is to be an input, output, analog or alternative function, determines that pin's setting (according to Table 2.3) in the corresponding field of the GPIOx_MODER register as shown in Figure 2.8.

Table 2.3 Bit patterns to select GPIO Mode.

MODER(i) [1:0]	I/O Configuration	Symbol Name in common-driver/gpio.h
00	Input	ESF_GPIO_MODER_INPUT
01	Output	ESF_GPIO_MODER_OUTPUT
10	Alternate Function	ESF_GPIO_MODER_ALT_FUNC
11	Analog	ESF_GPIO_MODER_ANALOG

We will use these MODER settings often, so in Listing 2.3 we define our own symbols with meaningful names which are easier to remember.¹ These symbols are defined in the file gpio.h in the common-driver folder of the code repository.

¹ In this textbook we use the prefix `ESF_` to distinguish these symbols from the symbols defined by ST in their support code. The hardware abstraction layer in `stm32f0xx_hal_gpio.h` defines similar values (`GPIO_MODE_...`), but adds other features and constraints, complicating the explanation of the key concepts.

```
#define ESF_GPIO_MODER_INPUT      (0)
#define ESF_GPIO_MODER_OUTPUT    (1)
#define ESF_GPIO_MODER_ALT_FUNC  (2)
#define ESF_GPIO_MODER_ANALOG    (3)
```

Listing 2.3 Descriptive names for MODER settings are defined in `common-driver\gpio.h`.

In order to access the GPIO MODER register with C code, we will use the CMSIS-CORE support from `stm32f091xc.h`. The data structure of type `GPIO_TypeDef` has its fields laid out to match the registers in a GPIO peripheral as shown in Listing 2.4. The `stm32f091xc.h` file also defines pointers called GPIOA, GPIOB, etc. to each GPIO module. We will use these pointers to access the data structures.

```
typedef struct
{
    __IO uint32_t MODER;    /*!< GPIO port mode register,
                           Address offset: 0x00 */
    __IO uint32_t OTyPER;   /*!< GPIO port output type register,
                           Address offset: 0x04 */
    __IO uint32_t OSPEEDR;  /*!< GPIO port output speed register,
                           Address offset: 0x08 */
    __IO uint32_t PUPDR;    /*!< GPIO port pull-up/pull-down register,
                           Address offset: 0x0C */
    __IO uint32_t IDR;      /*!< GPIO port input data register,
                           Address offset: 0x10 */
    __IO uint32_t ODR;      /*!< GPIO port output data register,
                           Address offset: 0x14 */
    __IO uint32_t BSRR;     /*!< GPIO port bit set/reset register,
                           Address offset: 0x1A */
    __IO uint32_t LCKR;     /*!< GPIO port configuration lock register,
                           Address offset: 0x1C */
    __IO uint32_t AFR[2];   /*!< GPIO alternate function low register,
                           Address offset: 0x20-0x24 */
    __IO uint32_t BRR;      /*!< GPIO bit reset register,
                           Address offset: 0x28 */
} GPIO_TypeDef;
```

Listing 2.4 C-language data structure representing GPIO port registers.

The CMSIS-CORE support in `stm32f091xc.h` includes these macros for accessing the MODERn field for pin n in the MODER register. We can use `GPIO_MODER_MODERn` definitions in Listing 2.1 to shift the desired value leftward to the field's position within the register.

Our switch and LED example system needs one input (the switch on PC13) and one output (LED LD2 on PA5). The MODER field of the GPIOA controls the connection of the pins to the GPIO module as inputs or outputs.

```
// Enable peripheral clock of GPIOA (for LD2)
RCC->AHBENR |= RCC_AHBENR_GPIOAEN;
// Configure PA5 in output mode (01=1)
MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER5, ESF_GPIO_MODER_OUTPUT);
// Enable peripheral clock of GPIOC (for Switch B1)
RCC->AHBENR |= RCC_AHBENR_GPIOCEN;
// Configure PC13 in input mode (00=0)
MODIFY_FIELD(GPIOC->MODER, GPIO_MODER_MODER13, ESF_GPIO_MODER_INPUT);
```

Listing 2.5 Using the `MODIFY_FIELD` macro to configure port control register multiplexers.

In order to preserve the other fields within the control register, we will use the `MODIFY_FIELD` macro defined previously to perform the read/modify/write operations: first zeroing out the bits for the `MODERN` field, then using OR to insert the new values.

There are up to eight alternate functions available for each pin; one is selected by a second multiplexer controlled by GPIO alternate function registers `GPIOx_AFRL` (for the pins 0 to 7 of the port x) and `GPIOx_AFRH` (for the pins 8 to 16 of the port x) shown in Figure 2.9.

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Field	AFSEL7				AFSEL6				AFSEL5				AFSEL4			
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Access	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	AFSEL3				AFSEL2				AFSEL1				AFSEL0			
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Access	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 2.9 `GPIOx_AFRL` register controls the second multiplexer to define the alternate function (if used).

The GPIOA register AFRL is made of 8 AFSEL fields which allow configuring the alternate function of pins 0 to 7 of the GPIOA. The register AFRH allows configuring the alternate function of pins 8 to 16 of the GPIOA. To specify the alternate function register for bit 6 of GPIOA, we would write `GPIOA->AFRL` using the `_VAL2FLD` macro and field `GPIO_AFRL_AFSEL6` to shift the value to the correct position and mask it.

The AFSELn control bits specify multiplexer behavior as shown in Table 2.4. Before writing to any GPIO register, be sure that the clock signal is provided to the port (using the `RCC_AHBENR` register described above). Otherwise, accessing it may have no effect, or it may trigger a fault and cause the processor to execute error handling code (e.g. for a hard fault).

Table 2.4 Alternate function selection field settings with examples for selected pins

AFSEL field value	Configuration	Pin PA1	Pin PA2	Pin PA5
000	Alternative 0	EVENTOUT	Timer TIM15_CH1	SPI1_SCK, I2S1_CK
001	Alternative 1	Serial Communication USART2_RTS	Serial Communication USART2_TX	Communication interface CEC
010	Alternative 2	Timer TIM2_CH2	Timer TIM2_CH3	Timer TIM2_CH1_ETR
011	Alternative 3	Touch sensing TSC_G1_IO2	Touch Sensing TSC_G1_IO3	Touch Sensing TSC_G2_IO2
100	Alternative 4	Serial Communication USART4_RX	No connection	No connection
101	Alternative 5	Timer TIM15_CH1N	No connection	
110	Alternative 6	No connection	No connection	No connection
111	Alternative 7	No connection	Comparator Output COMP2_OUT	No connection

If the mode of the GPIO is set to the alternate function mode then the pin function is chosen by the AFSEL field of the pin. The values (000 to 111) will have different effects for different pins. For details refer to Chapter 8 of the MCU's reference manual [2].

GPIO Peripheral

The STM32F0 GPIO peripheral module has six ports (GPIOA through GPIOF). Although each port could have up to 16 pins, there are not enough pins on the MCU package to support all of them. The MCU's data sheet pin-out section describes which port bits are implemented [3].

An input GPIO port bit lets us read a single bit digital value on an MCU pin. For example, if we connect the pin through a switch to ground, then the program can tell if the switch is open or closed. An output GPIO port bit enables the program to set an MCU pin to be a logic one or zero. For example, a program can light or extinguish an LED that is connected to an output pin.

The main logic hardware for a single GPIO port bit is simple and built around registers, as shown in Figure 2.10. Each GPIO port bit has its own version of this hardware. A register stores one bit of data, which is visible on the output signal (marked Q). A register will read its data input (marked D) and store that value every time its clock signal (marked with a triangle) is triggered. These signals are triggered when the CPU writes to or reads from the register's address, as labeled in Figure 2.10. The GPIO port bit has two main registers:

- Input data is held in the **data in register**. The MCU's I/O clock automatically updates this register (e.g. 48 million times per second). To read this register, the CPU reads from the Input Data Register address (GPIOx_IDR), which enables the buffer (triangle to the right of the register) to transmit its contents to the CPU over the data bus.

- Output data is held in the **data out register**. The CPU writes data to this register by putting the data on the data bus and writing to the Output Data Register address (GPIOx_ODR), which triggers the register's clock signal.

Multiple GPIO bits are grouped together to create a port that can be accessed in parallel for efficiency. MCU designers often make ports as wide as the processor's native data word size, so a 32-bit processor may have 32-bit wide ports. In the case of the STM32F0 MCU, each port has a maximum of 16 pins but the port registers are 32 bits wide.

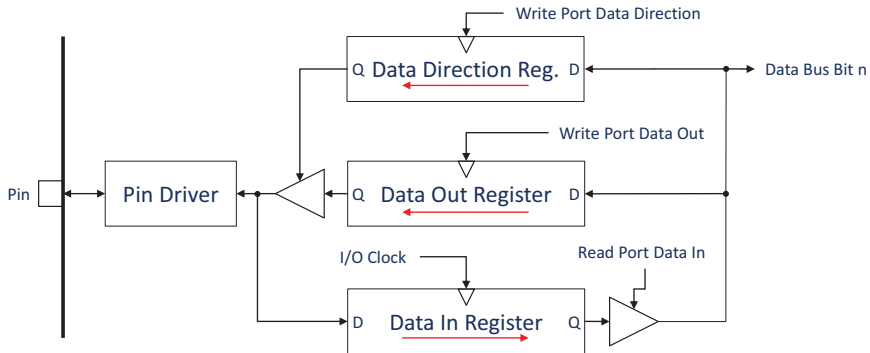


Figure 2.10 The main logic components of a GPIO port.

GPIO Module Use

In order to use the GPIO module, we need to read or write data:

- Input values are read from the GPIOx_IDR register.
- Output values are written to the GPIOx_ODR register.

```
// Definitions for bit positions
#define LD2_POS (5)
#define B1_POS (13)
#define MASK(x) (1UL<<(x))

// Turn off LD2
GPIOA->ODR &= ~MASK(LD2_POS);
while (1) {
    // Repeatedly read switch and control LD2
    if (GPIOC->IDR & MASK(B1_POS)) {
        // 1: Switch is not pressed, so turn off LD2
        GPIOA->ODR &= ~MASK(LD2_POS);
    } else {
        // 0: Switch is pressed, so turn on LD2
        GPIOA->ODR |= MASK(LD2_POS);
    }
}
```

Listing 2.6 Code excerpt to light LED when switch is pressed. GPIOA output for LED is controlled with read/modify/write instructions.

Listing 2.6 shows the code for the switch and LED example. There are first some definitions of bit positions and the **MASK** macro, and then code to turn off the LED. Then we run a loop that lights the LED only when the switch is held down.

In order to simplify and accelerate the software for manipulating individual output bits in a port, the hardware provides a special register `GPIOx_BSRR` (Bit Set and Reset Register) that can change specific output bits without affecting the others (that is, in an atomic way). Each bit *i* of the port has corresponding fields `BSi` and `BRi`. These are shown in Figure 2.11.

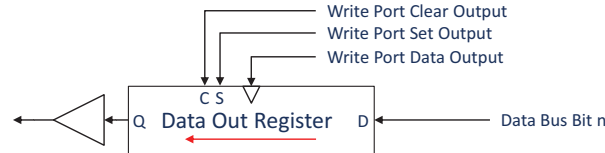


Figure 2.11 Additional control signals to clear and set output bits in an atomic way (without requiring separate read/modify/write operations).

- Writing a 1 to `GPIOx_BSRR_BSi` will set the corresponding `GPIO_ODRi`. For example, to set the least significant byte of port A to all ones, write `0x0000 00FF` to `GPIOA_BSRR` (`GPIOA->BSRR`). Writing the value 0 to the other bits has no effect.
- Writing a value 1 to `GPIOx_BSRR_BRi` will reset (clear) the corresponding `GPIO_ODRi`. For example, to clear the least significant byte of port A to all zeros, write `0x00FF 0000` to `GPIOA_BSRR` (`GPIOA->BSRR`). Writing the value 0 to the other bits has no effect.

```
// Turn off LD2
GPIOA->BSRR = GPIO_BSRR_BR_5;
while (1) {
    if (GPIOC->IDR & GPIO_IDR_13) {
        // 1: Switch is not pressed, so turn off LD2
        GPIOA->BSRR = GPIO_BSRR_BR_5;
    } else {
        // 0: Switch is pressed, so turn on LD2
        GPIOA->BSRR = GPIO_BSRR_BS_5;
    }
}
```

Listing 2.7 Updated code uses GPIO peripheral's port bit set and reset features (BS and BR) to simplify software read/modify/write operations to just writes.

Listing 2.7 shows the modified code which now lights the LED using the bit set and reset register. This simplifies the code, eliminating the need for read/modify/write instructions for each port access.

Putting the C Code Together

Now we can assemble the complete program, shown in Listing 2.8. We have defined additional symbols (`LD2_ON`, `LD2_OFF`, `B1_IN`) to make it easier to see how the I/O signals are connected and to take advantage of the port's bit set and reset features. The main function has one line of code which simply calls the example code. After downloading the code, press the reset button to start the program running.²

```
#define LD2_OFF_MSK      (GPIO_BSRR_BR_5)
#define LD2_ON_MSK       (GPIO_BSRR_BS_5)
#define B1_IN_MSK        (GPIO_IDR_13)

void Basic_Light_Switching_Example(void) {
    // Enable peripheral clock of GPIOA (for LD2)
    RCC->AHBENR = RCC_AHBENR_GPIOAEN;
    // Configure PA5 in output mode (01=1)
    MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER5, ESF_GPIO_MODER_OUTPUT);
    // Enable peripheral clock of GPIOC (for Switch B1)
    RCC->AHBENR = RCC_AHBENR_GPIOCEN;
    // Configure PC13 in input mode (00=0)
    MODIFY_FIELD(GPIOC->MODER, GPIO_MODER_MODER13, ESF_GPIO_MODER_INPUT);

    // Turn off LD2
    GPIOA->BSRR = LD2_OFF_MSK;
    while (1) {
        if (GPIOC->IDR & B1_IN_MSK) {
            // 1: Switch is not pressed, so turn off LD2
            GPIOA->BSRR = LD2_OFF_MSK;
        } else {
            // 0: Switch is pressed, so turn on LD2
            GPIOA->BSRR = LD2_ON_MSK;
        }
    }
}

int main(void) {
    Basic_Light_Switching_Example();
}
```

Listing 2.8 Completed code for LED and switch example.

More Interfacing Examples

Let's examine some more examples of interfacing: driving a three-color LED and a speaker.

² You can make the program run immediately after download: in MDK-Arm, select Options for Targets (Alt-F7), select the Utilities tab, click the Settings button to the right of Use Debug Driver, and in the Download Function section check the box Reset and Run. Then press OK and OK.

Driving a Three-Color RGB LED

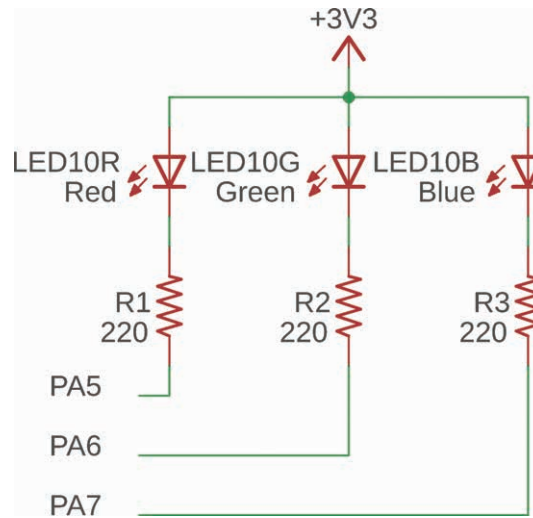


Figure 2.12 RGB LED connected to GPIO port bits through current-limiting resistors.

Let's see how to use three GPIO port bits from the Nucleo board to drive an external RGB LED. The schematic in Figure 2.12 shows that the three-color (red, green, blue or RGB) LED has its cathodes connected through resistors to the MCU through ports PA5 (red), PA6 (green), and PA7 (blue). The breadboard connections are shown in Figure 2.13.

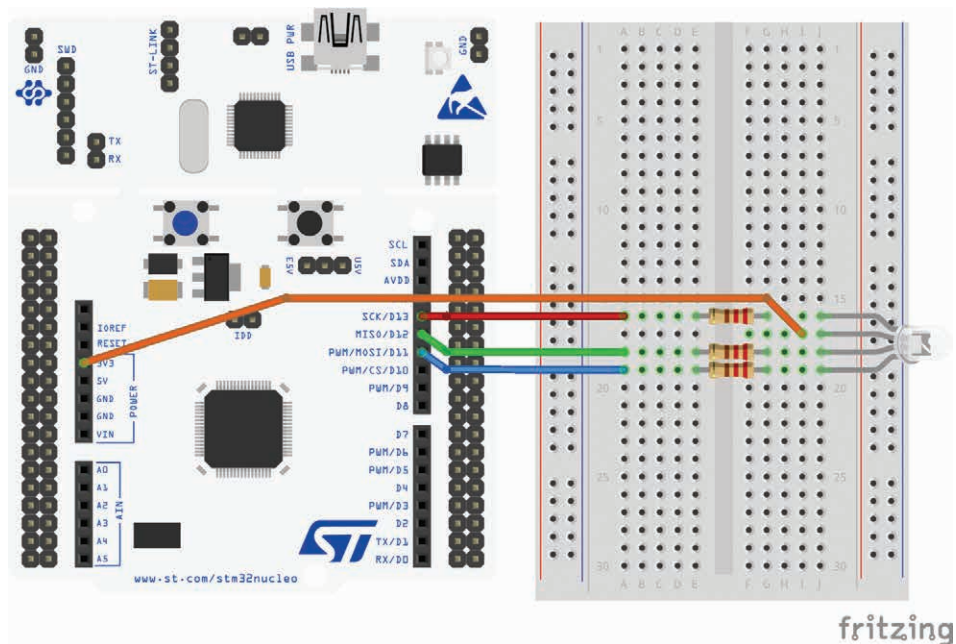


Figure 2.13 Layout of RGB LED circuit on breadboard.

Note that this is different from our previous example in Figure 2.1, in which the LED anode is connected to the MCU through a resistor. Because of this difference, clearing the output bit to zero will light the LED, and setting the output bit to one will turn off the LED. This configuration is used because many MCU output drivers can handle more current this way.

Let's examine a program to configure the port, lighting all possible combinations of the LEDs in a repeating sequence. The first part of the program defines useful values and initializes the peripherals, seen in Listing 2.9.

```
#include <stm32f091xc.h>
#include "delay.h"
#include "field_access.h"
#include "gpio.h"

// Active-Low LED control definitions
// 0 out = LED on
// 1 out = LED off
#define LED_R_OFF_MSK    (GPIO_BSRR_BS_5)
#define LED_R_ON_MSK     (GPIO_BSRR_BR_5)
#define LED_G_OFF_MSK    (GPIO_BSRR_BS_6)
#define LED_G_ON_MSK     (GPIO_BSRR_BR_6)
#define LED_B_OFF_MSK    (GPIO_BSRR_BS_7)
#define LED_B_ON_MSK     (GPIO_BSRR_BR_7)

void RGB_Flasher_Init(void) {
    // Enable peripheral clock of GPIOA
    RCC->AHBENR |= RCC_AHBENR_GPIOAEN;
    // Configure the three pins as output
    MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER5, ESF_GPIO_MODER_OUTPUT);
    MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER6, ESF_GPIO_MODER_OUTPUT);
    MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER7, ESF_GPIO_MODER_OUTPUT);

    // Turn on LEDs
    GPIOA->BSRR =
        LED_R_ON_MSK | LED_G_ON_MSK | LED_B_ON_MSK;
}
```

Listing 2.9 Definitions and initialization code for flashing RGB LED.

The function `RGB_Flasher_Sequential` in Listing 2.10 lights the LEDs based on tests of each bit in the variable `num`.

```
void RGB_Flasher_Sequential(void) {
    unsigned int num = 0;
    while (1) {
        num++;
        if (num & 1)
            GPIOA->BSRR = LED_R_ON_MSK;
        else
            GPIOA->BSRR = LED_R_OFF_MSK;
        if (num & 2)
            GPIOA->BSRR = LED_G_ON_MSK;
```

```

    else
        GPIOA->BSRR = LED_G_OFF_MSK;
    if (num & 4)
        GPIOA->BSRR = LED_B_ON_MSK;
    else
        GPIOA->BSRR = LED_B_OFF_MSK;
    Delay(400);
}
}

```

Listing 2.10 Code to flash RGB LED based on value of num variable.

In order to make the different LED colors visible to the human eye, we slow down the loop by calling a function (**Delay**) shown in Listing 2.11.

```

void Delay(volatile unsigned int time_del) {
    volatile int n;
    while (time_del--) {
        n = 1000;
        while (n--)
            ;
    }
}

```

Listing 2.11 Delay function slows down processor to make different LED colors visible. Function is located in `common-driver/delay.c`.

The function `RGB_Flasher_Sequential` performs three tests to determine whether to light the three LEDs. `RGB_Flasher_Parallel` eliminates these individual bit tests and writes the data in parallel to the LED, simplifying the code and saving time. The three least-significant bits of num are masked off and shifted to align with the GPIO port bits corresponding to the LEDs and then are written.

```

void RGB_Flasher_Parallel(void) {
    unsigned int num = 0;

    while (1) {
        num++;
        GPIOA->ODR &= ~((0x07) << 5); // Clear all bits in field
        GPIOA->ODR |= (~num & 0x07) << 5; // Set given bits in field
        // Note num is complemented with ~ so a 1 bit in num creates
        // a 0 output, turning on the LED
        Delay(400);
    }
}

```

Listing 2.12 Parallel version of RGB flasher writes all three bits from num to output port at once, simplifying code.

Driving a Speaker

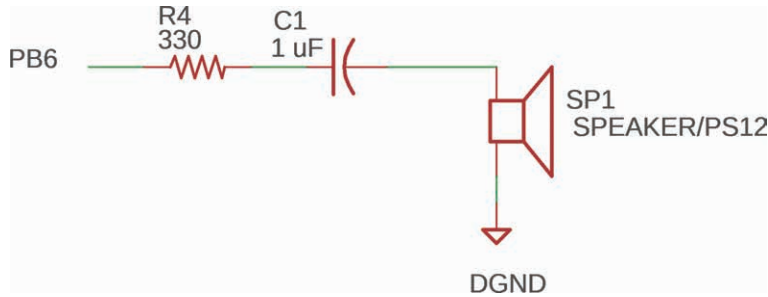


Figure 2.14 Driving a speaker with a digital output.

We can generate a simple tone with a speaker using the circuit shown in Figure 2.14. We use the MCU to toggle to an output PB6 to create a square wave that is filtered by a resistor and capacitor to drive a speaker. The frequency of the signal determines the pitch of the sound.

```
#include <stm32f091xc.h>
#include "delay.h"
#include "field_access.h"
#include "gpio.h"

#define SPKR_HIGH (GPIO_BSRR_BS_6)
#define SPKR_LOW (GPIO_BSRR_BR_6)

void Init_Speaker(void) {
    // Enable peripheral clock of GPIOB
    RCC->AHBENR |= RCC_AHBENR_GPIOBEN;
    // Configure Port B pin 6 as output (mode 1)
    MODIFY_FIELD(GPIOB->MODER, GPIO_MODER_MODER6, ESF_GPIO_MODER_OUTPUT);
    // Set to 1 initially
    GPIOB->BSRR |= SPKR_HIGH;
}

void Beep(void) {
    Init_Speaker();
    while (1) {
        // Delay is about 2 ms
        GPIOB->BSRR = SPKR_HIGH;
        Delay(2);
        GPIOB->BSRR = SPKR_LOW;
        Delay(2);
    }
}
```

Listing 2.13 Code to initialize speaker output and drive it with a square wave.

In this example, we use software and a delay loop to generate the square wave. The code in Listing 2.13 toggles the output by reading-inverting-writing the output register, and then delays for half of the period before repeating the process. We use the same time delay function as in the previous example, but with a shorter argument for a shorter delay.

The output waveforms are shown in Figure 2.15. The upper trace shows the digital output of the GPIO pin, toggling about every 2 ms (your times may vary). The lower trace shows the filtered voltage after the capacitor.

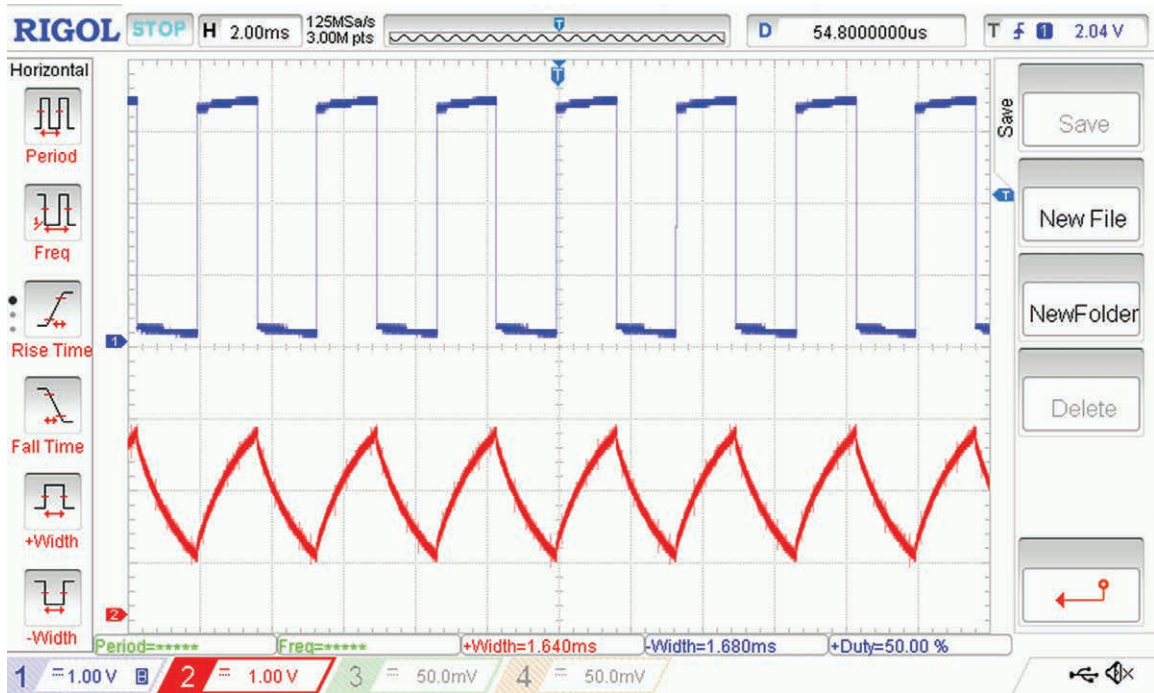


Figure 2.15 Waveforms generated by a beep function and speaker circuit.

Note that this software-based approach does not share the processor with the other processing tasks, so it limits what else the system can do during tone generation. Microcontrollers have a variety of other methods (peripherals such as timers and interrupt service routines) that can generate a square wave in hardware with minimal software overhead. We will be learning more about this in Chapter 7.

Driving the Hot Plate's Heating Element

In the first chapter we looked at how to use an embedded computer as a hot plate controller. The circuit requires a driver to switch the heating element and indicator light on and off. These devices require much more power and higher voltages than the MCU can provide. The heating element operates at 120 V, drawing about 8 A of current. This is far more than the MCU's digital output can provide. We need a driver circuit that uses a logic-level control signal (3.3 V, a few mA) to switch a mains-voltage level signal (120 V, 8 A).



Figure 2.16 A solid-state relay allows the MCU to switch devices powered by up to 380 V AC.

There are various circuits and devices that can be used. One convenient option is called a solid-state relay (SSR), shown in Figure 2.16. When a logic-level one is present on the input (terminals 3 and 4), the internal circuit will connect the output terminals (1 and 2) electrically. The input and output circuits in the SSR are completely electrically isolated to make the circuit safer, reducing the risk of circuit damage or user injury from voltage surges on the mains or circuit malfunctions.

Additional Pin Configuration Options

The MCU may offer additional options for the circuits driving each output pin or sensing each input pin. We examine the most common options here.

Pull-Ups and Pull-Downs for Inputs

Some types of input signals will not swing the full voltage range from valid logic zero to valid logic one. For example, the switch in Figure 2.1 can only pull the input signal to ground. If the switch is not pressed, the input signal will be disconnected (“floating”) and sensitive to electrical noise (e.g. static electricity). The resistor R30 was added to pull the signal up to V_{DD} when the switch is open.

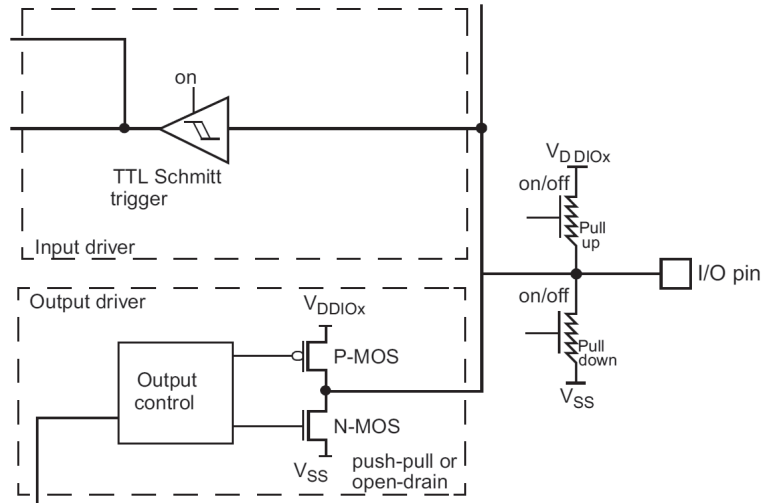


Figure 2.17 Input and output drivers, and pull-up and pull-down circuits [2].

This pull functionality is so useful that most microcontrollers include built-in pull-up and pull-down support; STM32F0 MCUs are no exception. To simplify the circuit design, weak transistors are typically used as controllable resistors, as shown on the right side of Figure 2.17. The typical value of the pull-up and pull-down resistance is between 20 and 50 k Ω .

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Field	PUPDR15		PUPDR14		PUPDR13		PUPDR12		PUPDR11		PUPDR10		PUPDR9		PUPDR8	
Reset	0	0	x	0	0	x	0	0	0	0	0	0	0	0	0	0
Access	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	PUPDR7		PUPDR6		PUPDR5		PUPDR4		PUPDR3		PUPDR2		PUPDR1		PUPDR0	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Access	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 2.18 STM32F0 pin pull-up/pull-down register (GPIOx_PUPDR) contents for ports B to F. Port A is similar but has different reset values [2].

We configure the pull functionality using the pull-up/pull-down control register (PUPDR) shown in Figure 2.18. Two bits are associated in the registers to each pin of the port. It controls which circuits are enabled: either the pull-up (01), pull-down (10) or neither (00). After reset all pull resistors are disabled for ports B through F, while port A enables some [2].

Using this feature, we could simplify the circuit of Figure 2.1 by eliminating the external pull-up resistor. This would reduce parts and assembly costs, as well as circuit size. The code would need to enable the pull-up resistor by setting the bits in PUPDR corresponding to the input pins to 01.

Open Drain vs. Push-Pull Outputs

Normal digital outputs use two transistors to switch the signal quickly between voltages V_{DD} (for 1) and ground (for 0) as seen in the output driver section of Figure 2.17. Sometimes we wish to combine multiple outputs to a single signal, for example to enable multiple devices to drive a signal such as an error condition, or a communication bus (we will see the I²C communication protocol uses this approach in Chapter 8). If we connect multiple push-pull outputs together, and they have different data to send, the driver transistors will be damaged. The output driving a 1 will turn on its P-MOS transistor, and the output driving a 0 will turn on its N-MOS transistor. The resulting low-resistance path from V_{DD} to ground will let too much current to flow through the P-MOS transistor in one output and the N-MOS transistor in the other output, damaging and even destroying them.

To prevent this damage, we can use open-drain drivers: the P-MOS transistor is disabled regardless of whether a 1 or 0 is sent. To send a 1, the N-MOS transistor is turned off. A pull-up resistor (instead of the P-MOS transistor) pulls the signal up to 1. To send a 0, the N-MOS transistor is turned on, and a small but safe amount of current will flow through that transistor. The output changes from a 0 to a 1 more slowly than with a P-MOS transistor. “Open-drain” means that the drain terminal of the N-MOS transistor is not connected to a working P-MOS transistor.

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Field	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
Reset																
Access																

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Access	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 2.19 Output type register GPIOx_OTYPER determines if each output is push-pull(0, default after reset) or open-drain (1).

The GPIO port output type register (GPIOx_OTYPER) in Figure 2.19 controls the type of the output between open-drain and push-pull. Note that GPIO port pull-up/pull-down register (GPIOx_PUPDR) is used for both inputs and outputs. In the case of an open-drain output, we want to use the pull-up setting (01) in GPIOx_PUPDR if there is no external pull-up on the signal.

Output Drive Strength

Digital output signals generate electromagnetic (EM) noise as they change. For some applications fast changes are more important, while for others EM noise dominates. The GPIO peripheral offers three output transition speeds (low, medium and high speed) which are selected by the GPIOx_OSPEEDR register, described in the GPIO port output speed register section of the MCU reference manual [2]. The resulting signal transition speeds are described in the Input/Output AC characteristics section of the MCU data sheet [3].

Configuration Lock

It is possible to lock the configuration (e.g. direction, pull-up) of bits in ports A and B to prevent corruption by software bugs or glitches. After locking, only the bit's data can be changed; the configuration is fixed until the MCU is reset. Details appear in the GPIO port configuration lock register section of the reference manual [2].

Other Options

The GPIO peripheral can generate events such as interrupt and direct memory access requests based on input signals. These will be discussed in Chapters 4 and 9 and are controlled by the EXTI and SYSCFG registers.

Summary

This chapter has introduced the concepts of general purpose I/O ports and how to use them in a STM32F0 MCU using C code with the CMSIS-CORE hardware abstraction layer or the STMicroelectronics HAL hardware abstraction layer. Interfacing with inputs and outputs was introduced with examples using LEDs, a switch, and a speaker.

Exercises

1. What are the valid input voltage ranges for a STM32F0 MCU with $V_{DD} = 3\text{ V}$? With 2 V ?
2. What is the actual value of V_{DD} on your MCU board? Use a multimeter to measure this voltage.
3. Examine the schematic for your Nucleo-F091RC board:
 - a. How many GPIO port bits are available for port A?
 - b. Port B?
 - c. Port C?
 - d. Port D?
 - e. Port E?
 - f. Port F?
4. Calculate the resistor values needed to limit current through the blue and red LEDs of Figure 2.3 to 18 mA each. Assume that the supply voltage V_{DD} is 3.0 V .
5. What values need to be written to which registers in order to set a port as a digital input, with the resistive pull-up enabled?
6. Consider a program that uses bits 0 through 5 on port A as GPIO inputs, and bits 10 through 15 as GPIO outputs:
 - a. What control register settings are needed?
 - b. Write a C code to implement these control register settings.

7. Consider a system with a STM32F091RC MCU, one switch, and four LEDs. As long as the switch is not pressed, all lights shall be turned off. When the switch is pressed, the LEDs shall start to turn on one at a time (starting with LED 0), with a delay of roughly $\frac{1}{2}$ second per LED. After all the LEDs are turned on, they shall remain on until the switch is released. The selection of port bits for the switch and the LEDs is your choice. The following table shows the required sequence of LED activity, assuming the switch is pressed at time T_1 and released at time T_2 .

Time (sec)	Switch	LED0	LED1	LED2	LED3
$< T_1$	Not pressed	off	off	off	off
T_1	Pressed	on	off	off	off
$T_1 + 0.5$	Pressed	on	on	off	off
$T_1 + 1.0$	Pressed	on	on	on	off
$T_1 + 1.5$	Pressed	on	on	on	on
T_2	Not pressed	off	off	off	off

- What control register settings are needed?
- Write a C program to implement this system.

References

- [1] STMicroelectronics NV, *User Manual UM1724: STM32 Nucleo-64 Boards*, 2019.
- [2] STMicroelectronics NV, *Reference Manual RM0091: STM32F0x1/STM32F0x2/STM32F0x8*, 2017.
- [3] STMicroelectronics NV, *STM32F091xB STM32F091xC Data Sheet*, DocID 026284, 2017.

